NeuroSlide: A Pipelined CNN Hardware Accelerator for Efficient Image Classification

SOC PROJECT

Venkata Durgesh-230907454

Gurudeep Sai-230907202

Sai Suneeth-230907176

Manudeep-230907436

ABSTRACT

Convolutional Neural Networks (CNNs) are widely used for image classification tasks but require high computational resources, particularly in convolution operations. This project proposes NeuroSlide, a custom-designed CNN hardware accelerator aimed at improving computational efficiency through parallelism and pipelining.

The system processes input images by converting them to grayscale, applying padding, and feeding them into a multi-layer CNN architecture. The network consists of two convolutional layers with ReLU activation and max-pooling, followed by a fully connected Artificial Neural Network (ANN) classifier. The architecture is designed using 8-bit precision to optimize hardware resource usage.

A key contribution of this work is the design of a three-stage pipelined architecture combined with parallel convolution units, enabling simultaneous processing of multiple feature maps. Additionally, a systolic array-based approach is proposed for accelerating convolution operations by performing parallel multiply-accumulate computations with reduced memory access overhead.

The proposed design targets efficient real-time image classification applications such as pothole detection, with future work focusing on detailed hardware implementation and optimization of the systolic array module.

Introduction

In recent years, Convolutional Neural Networks (CNNs) have become a fundamental component in image processing and computer vision applications such as object detection, medical imaging, and road condition monitoring. Their ability to automatically extract hierarchical features from raw image data makes them highly effective for classification tasks. However, this performance comes at the cost of **high computational complexity**, particularly in convolution operations that involve a large number of multiply-accumulate (MAC) computations.

Traditional implementations of CNNs on general-purpose processors face challenges in terms of **latency, power consumption, and memory bandwidth**, making them inefficient

for real-time and embedded applications. This has led to increasing interest in **hardware accelerators** that can optimize CNN execution through parallelism and efficient data handling.

This project presents **NeuroSlide**, a custom-designed CNN hardware accelerator architecture aimed at improving computational efficiency through structured dataflow and parallel processing. The proposed system introduces a **pipelined architecture**, where different stages of computation—convolution, activation, pooling, and classification—are executed in parallel to increase throughput. Additionally, the design incorporates **parallel convolution units**, enabling simultaneous processing of multiple feature maps.

A key focus of this work is optimizing the most computationally intensive part of CNNs: the convolution operation. To address this, a **systolic array-based approach** is proposed for efficient matrix multiplication, allowing parallel execution of multiply-accumulate operations while minimizing memory access overhead.

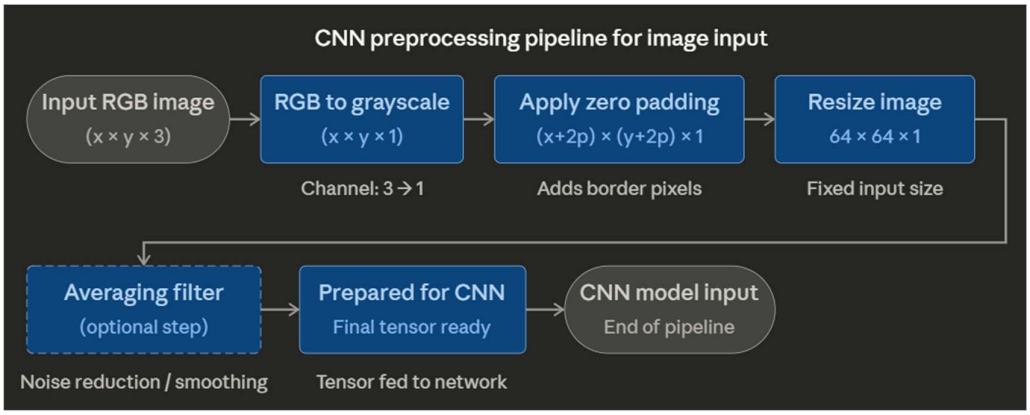
The proposed architecture is applied to an image classification task, specifically **road condition analysis**, where the system classifies images into categories such as small potholes, large potholes, and normal road surfaces. The design emphasizes both performance and hardware feasibility by using reduced precision (8-bit data representation) and modular pipeline stages.

This report details the system architecture, data flow, and design considerations of the NeuroSlide accelerator, with further work focused on implementing and optimizing the convolution engine using systolic array structures.

System Overview

2.1 Input Processing

The system accepts an RGB image as input, which is first converted into a grayscale representation to reduce computational complexity. The image is then resized to a fixed dimension of 64×64 pixels and padded appropriately to preserve spatial dimensions during convolution operations. Each pixel is represented using 8-bit precision, ensuring a balance between hardware efficiency and accuracy.

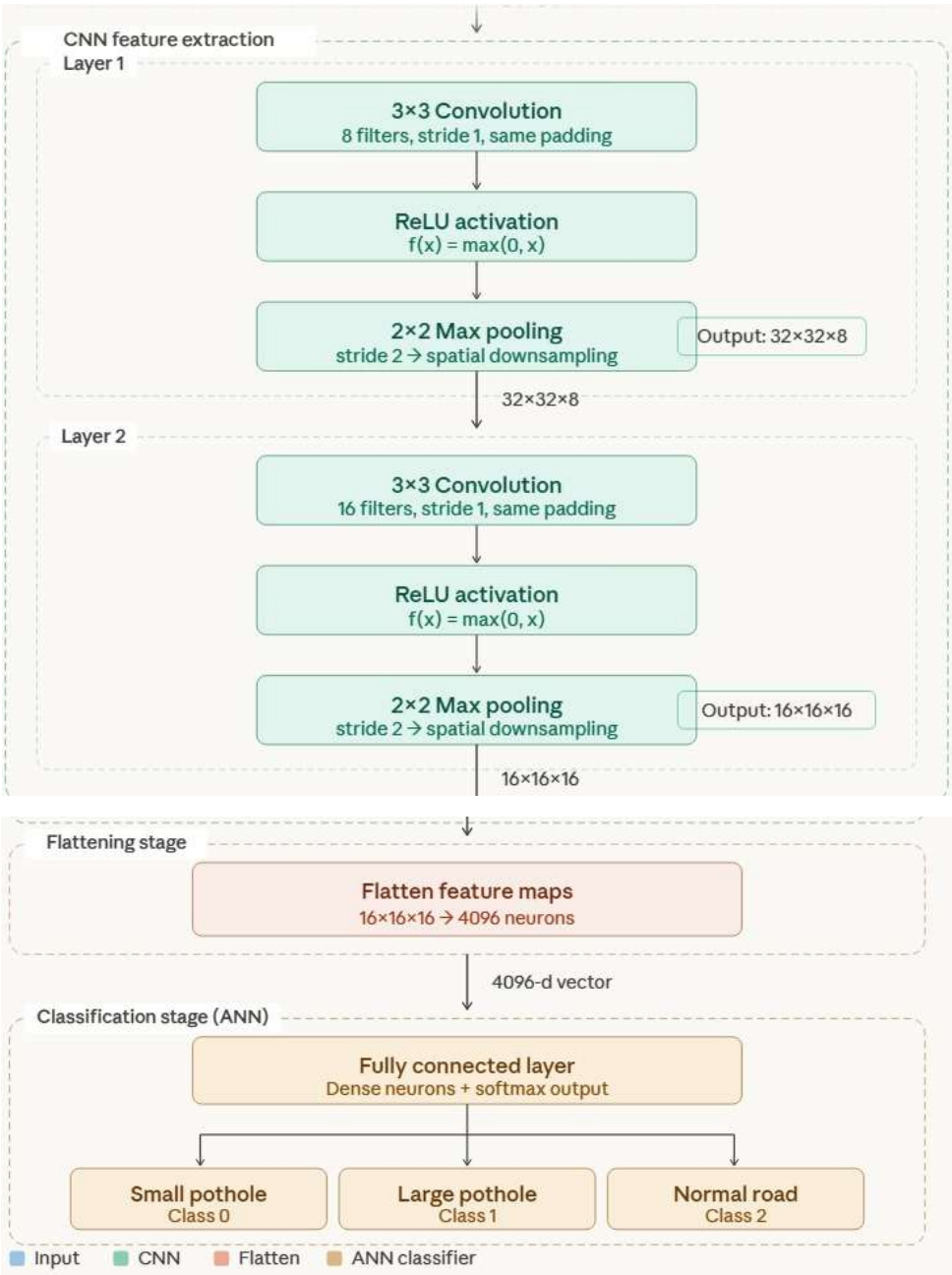


2.2 CNN Processing Pipeline

The processed image is fed into a CNN-based feature extraction pipeline consisting of multiple convolutional layers. Each layer performs:

- 3×3 convolution
- ReLU activation
- 2×2 max pooling

The architecture is designed to progressively extract spatial features while reducing dimensionality. The feature maps generated after successive layers are then flattened and passed to a fully connected ANN classifier.



2.3 Hardware-Oriented Convolution Design

A key aspect of the system is the hardware implementation of the convolution operation using a streaming dataflow architecture. The convolution engine is designed based on line buffers and a sliding window mechanism, as demonstrated in the developed Verilog-style code .

1. **Line Buffering Mechanism**

   Incoming pixel data is stored in multiple line buffers, allowing reuse of previously fetched data. This avoids redundant memory access and enables continuous data streaming.

2. **Sliding Window Generation**

   A 3×3 window is dynamically formed using data from the line buffers. The window shifts across the image in a clock-driven manner, ensuring efficient coverage of all pixels.

3. **Multiply-Accumulate Operation**

   For each window position, the system performs parallel multiplication with kernel weights followed by accumulation:

   - 9 multiplications (for 3×3 kernel)
   - 1 summation output per cycle (after pipeline fill)

4. **Output Handling**

   Valid convolution outputs are stored in memory (register/BRAM), controlled by synchronization signals such as sum_valid and done_writing. This ensures correct timing and data integrity in the pipeline.

2.4 Parallel and Pipelined Architecture

The system is designed to support parallel processing of multiple convolution filters, enabling simultaneous generation of feature maps. Additionally, a multi-stage pipeline architecture is used, where different operations (convolution, activation, pooling) are executed concurrently across stages.

This combination of data streaming, pipelining, and parallelism significantly improves throughput and makes the architecture suitable for real-time applications.

2.5 Output Classification

The extracted features are flattened and passed to an Artificial Neural Network (ANN) classifier. The system outputs classification results for predefined categories such as:

- Small pothole
- Large pothole
- Normal road

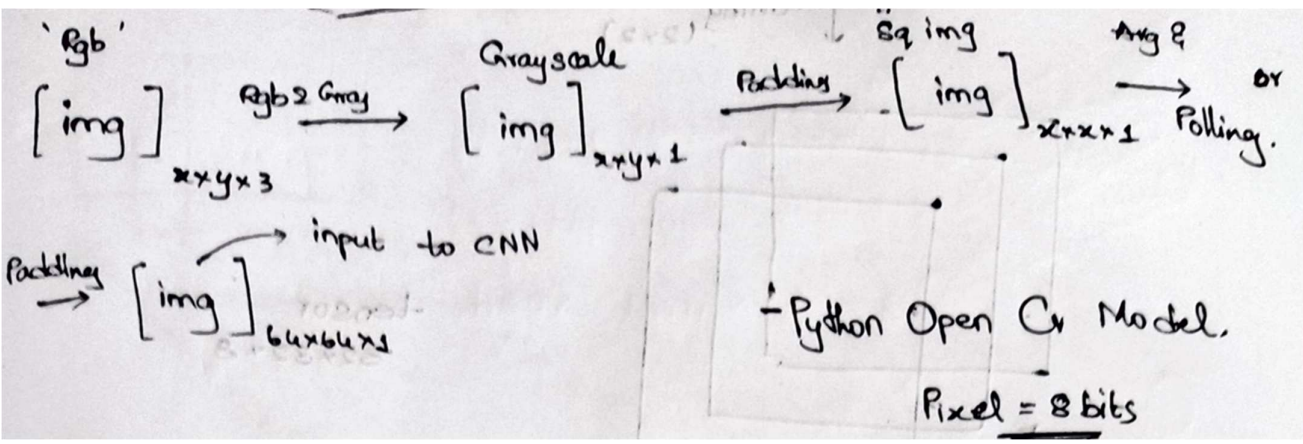
Proposed CNN Architecture

The proposed Convolutional Neural Network (CNN) architecture in NeuroSlide is designed to efficiently extract spatial features from input images while maintaining hardware feasibility. The architecture consists of multiple convolutional layers followed by a fully connected Artificial Neural Network (ANN) classifier. Each stage is carefully structured to balance feature extraction capability and computational efficiency.

3.1 Input Layer

The input to the system is a preprocessed grayscale image of size 64×64 pixels. The preprocessing stage includes RGB to grayscale conversion and padding to preserve spatial dimensions during convolution.

Each pixel is represented using 8-bit precision, enabling efficient hardware implementation while maintaining sufficient image detail.



3.2 Convolutional Layer 1

The first convolutional layer is responsible for extracting low-level spatial features such as edges and textures.

- Kernel Size: 3×3
- Number of Filters: 8
- Input Dimension: 64×64×1
- Output Dimension: 64×64×8

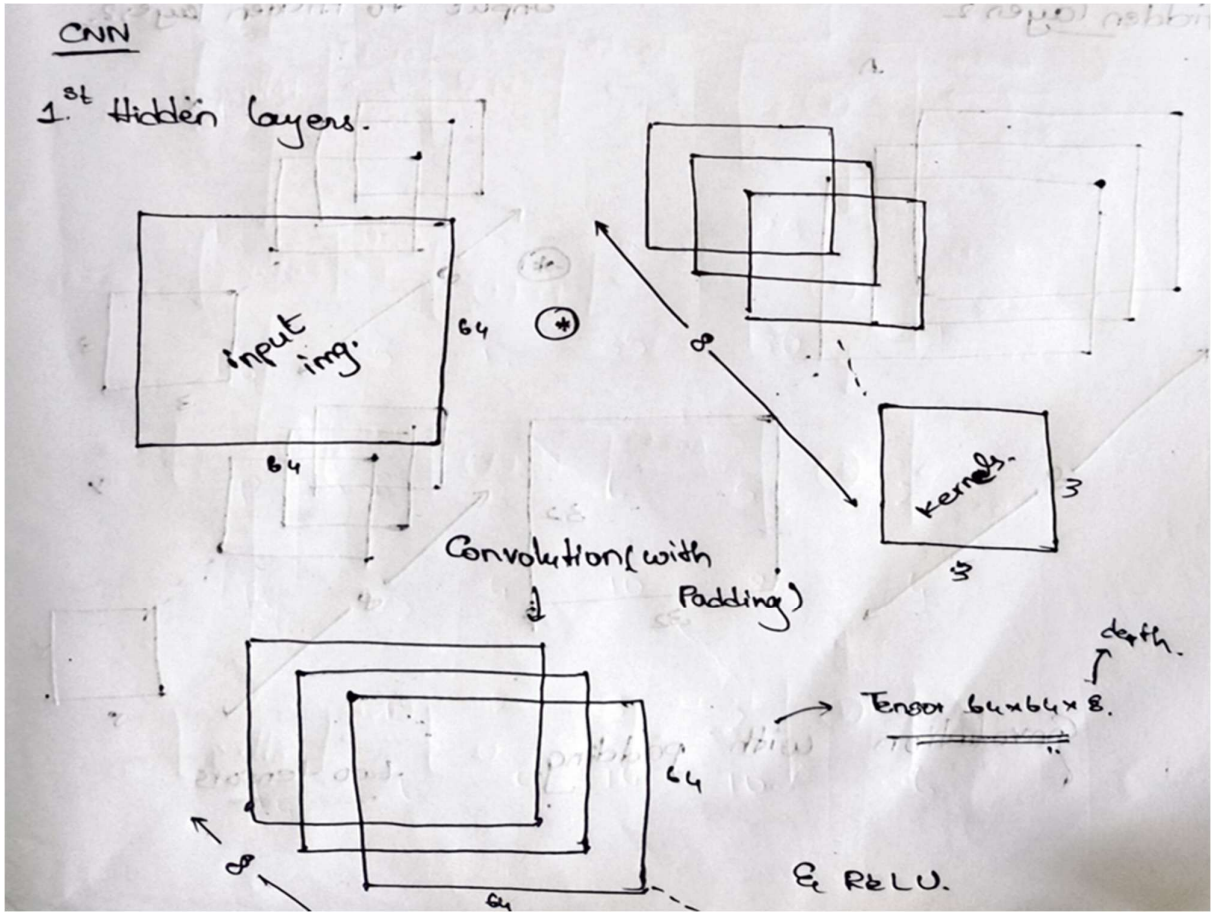
Each filter performs convolution across the input image to generate a feature map. This is followed by:

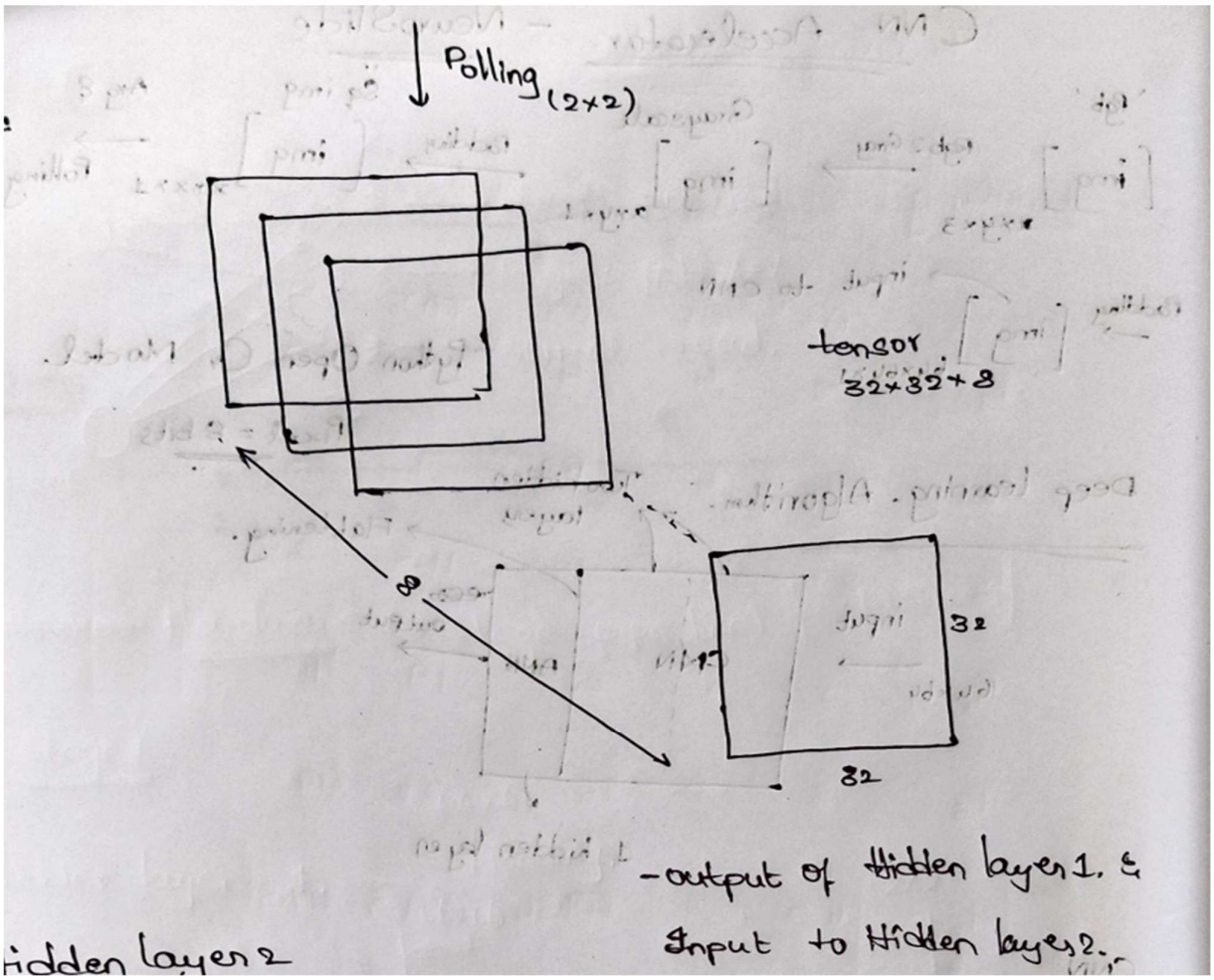
- ReLU Activation Function to introduce non-linearity
- 2×2 Max Pooling to reduce spatial dimensions

After pooling, the output becomes: 32×32×8

CNN

1st Hidden layers.





3.3 Convolutional Layer 2

The second convolutional layer extracts higher-level features by operating on the output of the previous layer.

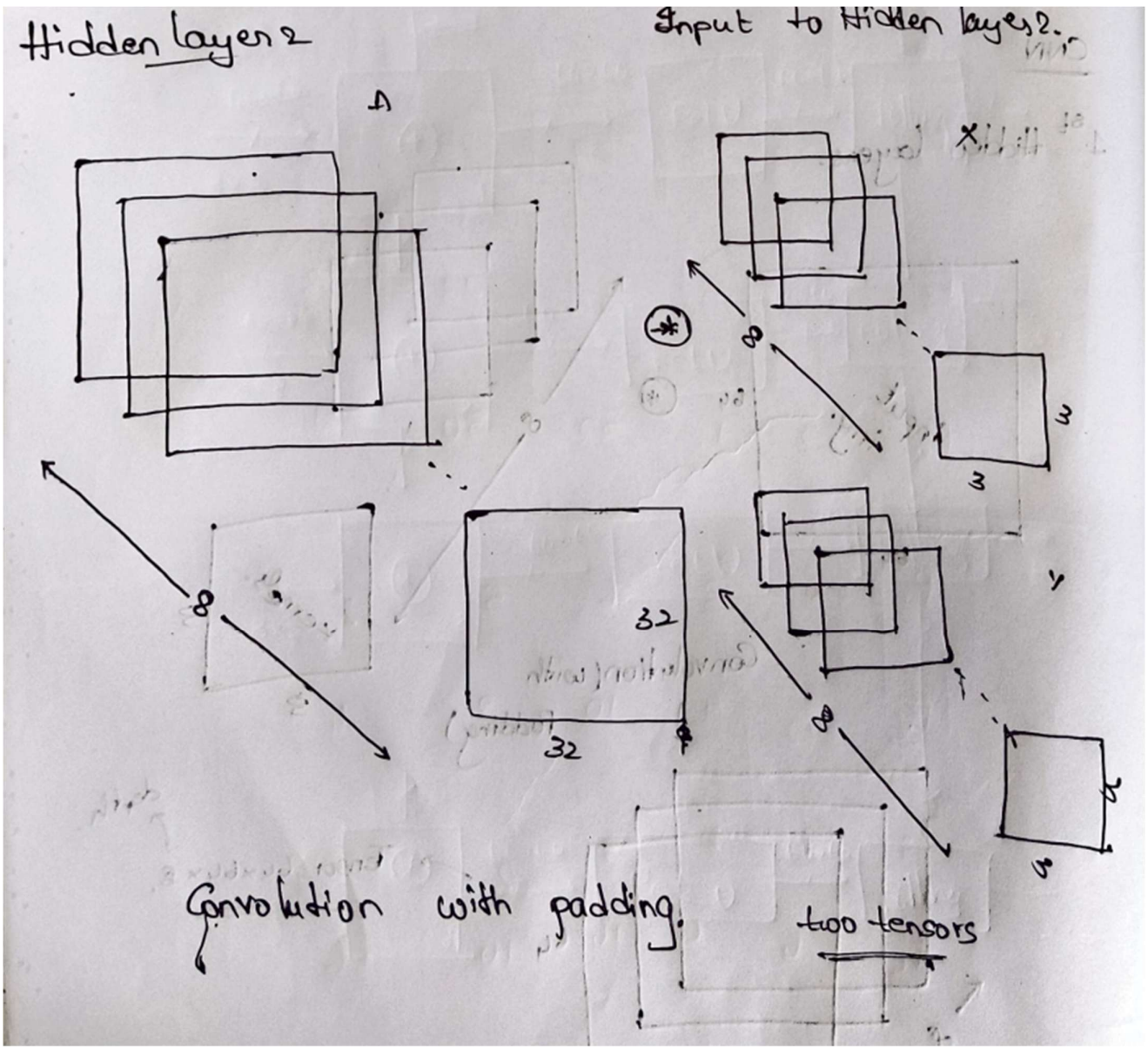
- Kernel Size: 3×3
- Number of Filters: 16
- Input Dimension: 32×32×8
- Output Dimension: 32×32×16

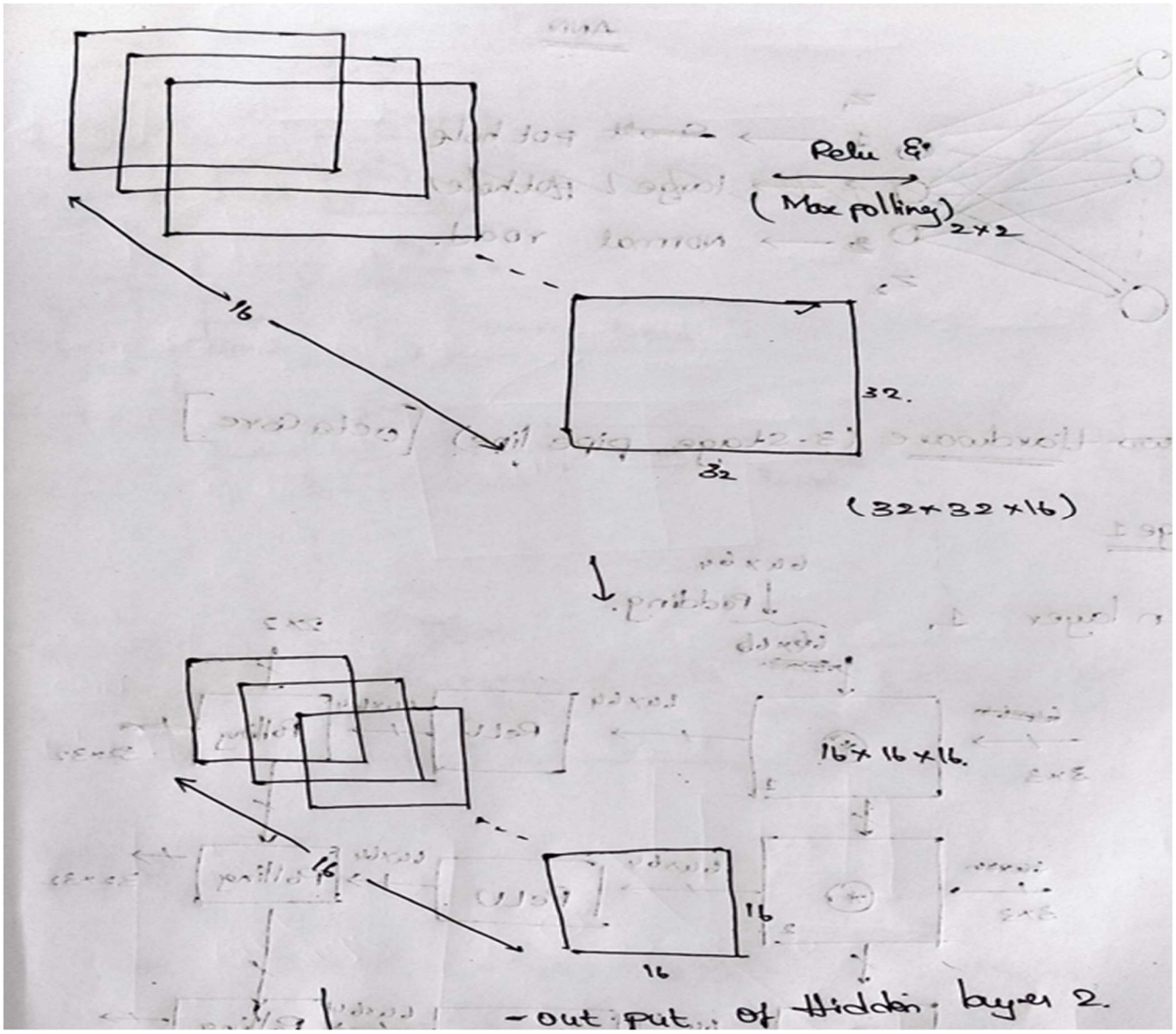
Similar to the first layer, this is followed by:

- ReLU Activation
- 2×2 Max Pooling

After pooling, the output becomes:

- 16×16×16



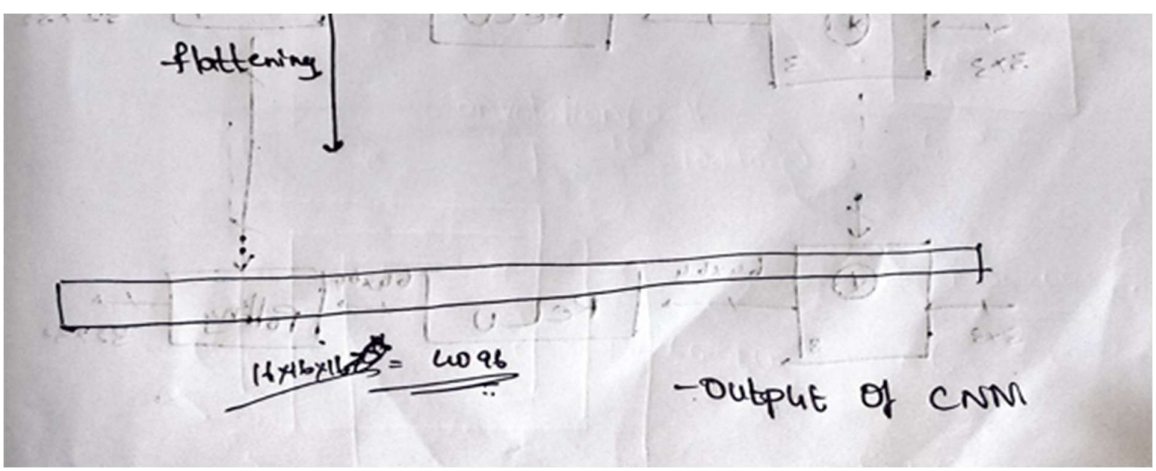


3.4 Feature Flattening

The output feature maps from the second convolutional layer are flattened into a one-dimensional vector to serve as input to the classifier.

- Input: **16×16×16**
- Flattened Output: **4096 neurons**

This transformation allows the spatial features to be processed by fully connected layers.



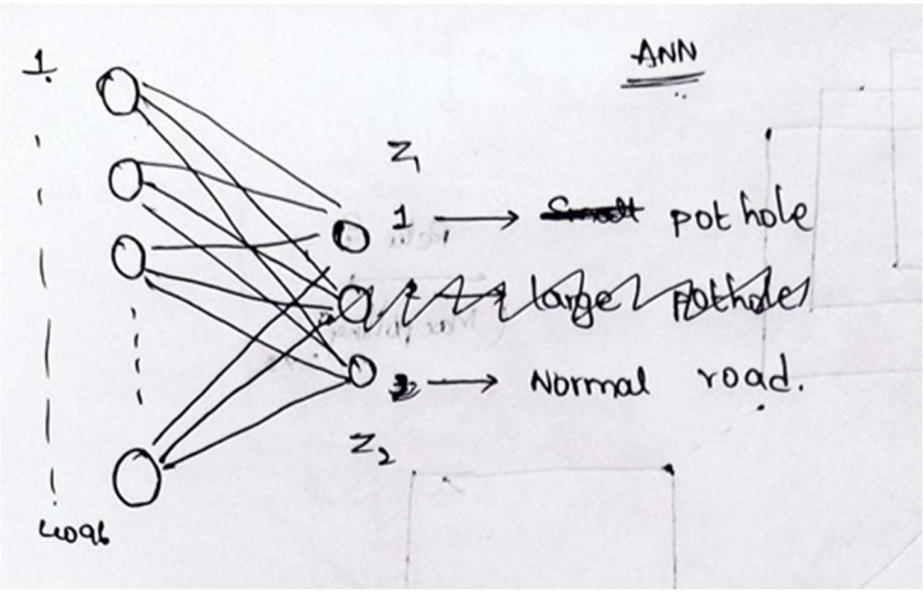
3.5 Fully Connected ANN Classifier

The flattened feature vector is fed into an Artificial Neural Network (ANN) for classification.

The classifier processes the extracted features and produces output corresponding to predefined categories:

- Small pothole
- Large pothole
- Normal road

The ANN performs weighted summation of inputs followed by activation to generate the final classification output.



3.6 Design Considerations

The architecture is designed with the following considerations:

- **Computational Efficiency:** Use of small kernel size (3×3) reduces computation while maintaining feature extraction capability
- **Dimensionality Reduction:** Pooling layers reduce data size, lowering memory and computation requirements
- **Hardware Compatibility:** Fixed input size and 8-bit precision support efficient hardware implementation
- **Scalability:** The modular design allows extension to deeper networks if required

HARDWARE ARCHITECTURE

6.1 Pipeline Design

The proposed architecture is implemented using a 3-stage pipeline to improve processing efficiency and throughput.

Stage	Operation
Stage 1	Convolution + ReLU + Pooling
Stage 2	Deeper Convolution Layers
Stage 3	ANN Classification

- In Stage 1, input image pixels are processed using 3×3 convolution filters, followed by ReLU activation and pooling.
- In Stage 2, deeper convolution layers extract higher-level features from the intermediate feature maps.
- In Stage 3, the extracted features are fed into an Artificial Neural Network (ANN) for final classification.
- The pipeline allows multiple data samples to be processed simultaneously at different stages, thereby improving throughput through parallel processing.

6.2 Parallel Convolution Engine

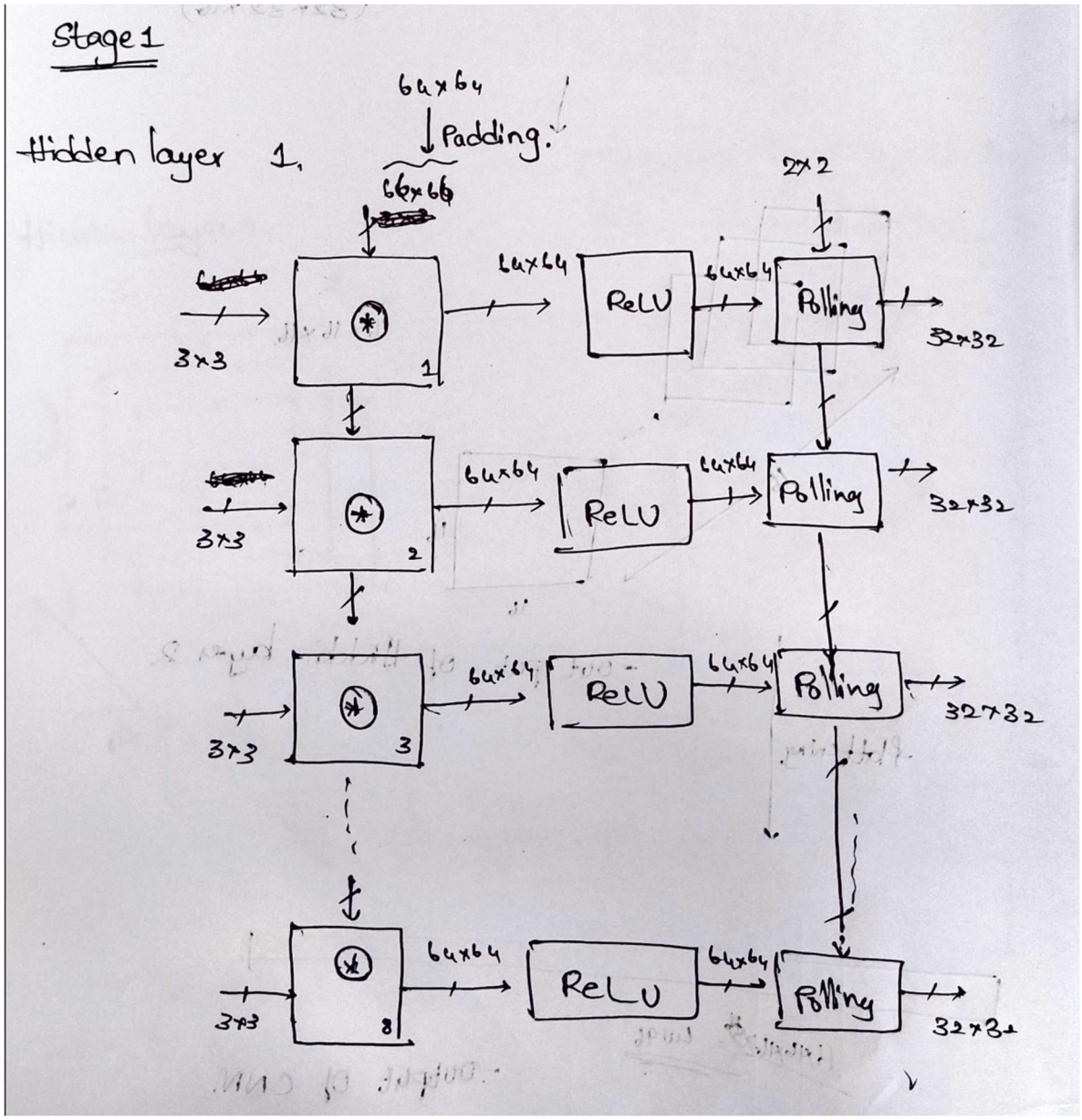
The convolution operation is implemented using a parallel processing architecture.

- Multiple 3×3 filters operate simultaneously on the input feature map.
- Each filter has an independent computation path, consisting of multiply-accumulate (MAC) units.
- This enables simultaneous generation of multiple feature maps.
- Spatial parallelism is utilized to compute multiple convolution outputs concurrently, significantly reducing computation time.

6.3 Data Flow Architecture

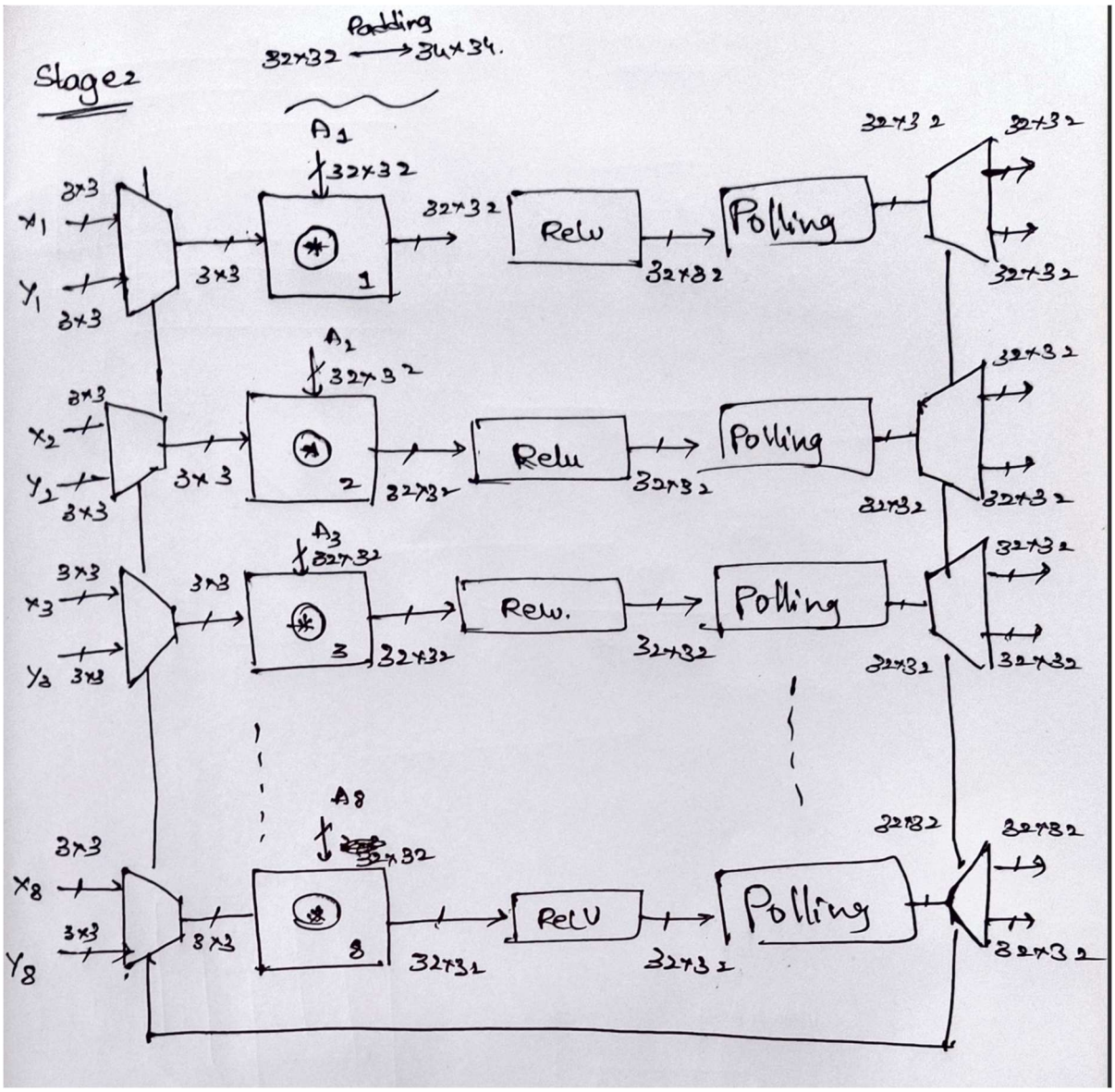
The architecture follows a stream-based data flow model.

- Input data is divided into multiple parallel streams (e.g., x_1, x_2, …, x_8).
- Each stream is processed independently through:
  1. Convolution
  2. ReLU activation
  3. Pooling
- The outputs from these parallel paths are then combined and forwarded to the next stage.
- Data is processed in parallel pipelines and merged for subsequent layers, ensuring continuous data flow and high hardware utilization.



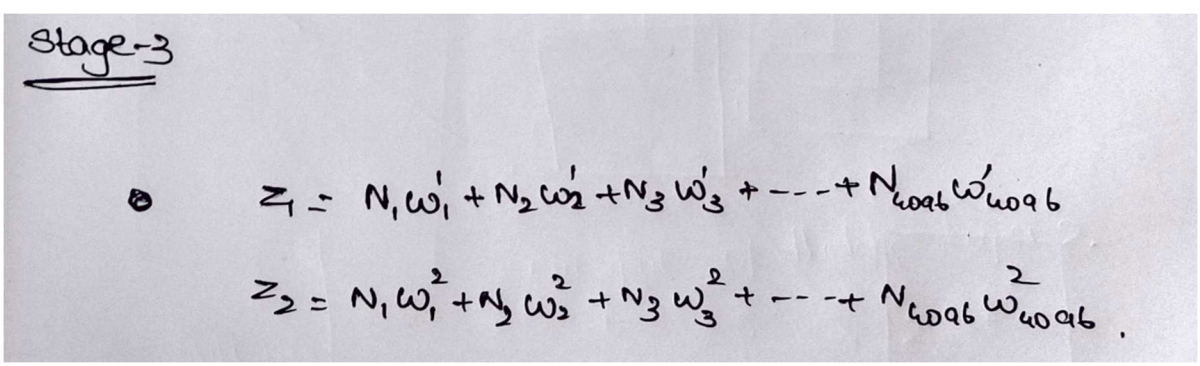
Stage 1: Parallel Convolution Layer

- The input image (64×64) is first padded and fed into multiple 3×3 convolution filters.
- Each filter operates in parallel, producing separate feature maps.
- ReLU activation is applied to introduce non-linearity.
- A 2×2 pooling operation reduces the spatial dimensions to 32×32.



Stage 2: Multi-Channel Convolution

- The output from Stage 1 is divided into multiple channels.
- Each channel is processed independently using 3×3 convolution filters.
- ReLU and pooling operations are again applied.
- Outputs are combined using a merging mechanism for further processing.



Stage 3: ANN Computation

- The extracted features are flattened and passed to the ANN.
- Each neuron computes a weighted sum of inputs:
- The ANN performs classification based on learned weights.

6.4 Memory Considerations

Efficient memory design plays a crucial role in achieving high performance in CNN hardware acceleration.

- An input buffer is used to store incoming image pixel data.
- Intermediate feature maps are stored between pipeline stages for further processing.
- A streaming-based architecture is adopted to reduce redundant memory access and improve data reuse.

BRAM-Based Memory Implementation

The design utilizes on-chip Block RAM (BRAM) for both input storage and intermediate result storage to achieve low latency and high throughput.

1. Input BRAM (Pixel Storage)

- The input image pixels are stored in BRAM.
- Pixels are accessed sequentially using an address counter.
- This enables continuous streaming of data into the convolution pipeline.

RTL Implementation:

```
blk_mem_gen_0 bram (        // BRAM instance
   .clka(clk),
   .ena(1'b1),
   .wea(1'b0),
   .addra(addr),
   .douta(pixel)
);
```

Explanation:

- addra → Address used to fetch pixel data
- douta → Output pixel from memory
- wea = 0 → Read-only operation
- Acts as the input image memory

2. Output BRAM (Feature Map Storage)

- The output of the convolution operation is stored in a separate BRAM.
- This memory stores the feature maps generated by the convolution layer.
- Data is written only when the output is valid.

RTL Implementation:

```
stage1_kernel_op kernel1(          // stores output of kernel 1
    .clka(clk),
    .ena(1'b1),
    .wea(sum_valid),
    .addra(adk),
    .dina(sum[7:0]),
    .douta(dout)
  );
```

Explanation:

- dina → Input data (convolution result)
- wea = sum_valid → Write enabled only for valid outputs
- addra → Address for storing output data
- Stores convolution results (feature maps)

Key Advantages of Using BRAM

- Provides low-latency and high-speed access
- Enables efficient streaming of data
- Reduces dependency on external memory
- Supports pipelined and parallel processing architectures

CONVOLUTION AND COMPUTATION OPTIMIZATION

Convolution is the most computationally intensive operation in a Convolutional Neural Network (CNN).

- It involves a large number of repeated multiply-accumulate (MAC) operations.
- For every output pixel, multiple multiplications and additions are required.
- This leads to high computation time and increased memory access.

Therefore, optimizing convolution is essential for improving overall system performance.

7.1 Proposed Optimization

To address this bottleneck, a systolic array-based architecture is proposed.

- The systolic array performs multiple MAC operations in parallel.
- Data flows rhythmically through processing elements (PEs), enabling continuous computation.
- Intermediate results are reused locally, reducing unnecessary data movement.

A systolic array-based architecture is proposed to perform parallel multiply-accumulate operations, significantly improving computational efficiency and reducing memory access overhead.

7.2 Why Systolic Array

The systolic array architecture is chosen due to the following advantages:

1. Data Reuse

- Input data and weights are reused across multiple processing elements.

- Minimizes redundant memory access.

2. Parallelism

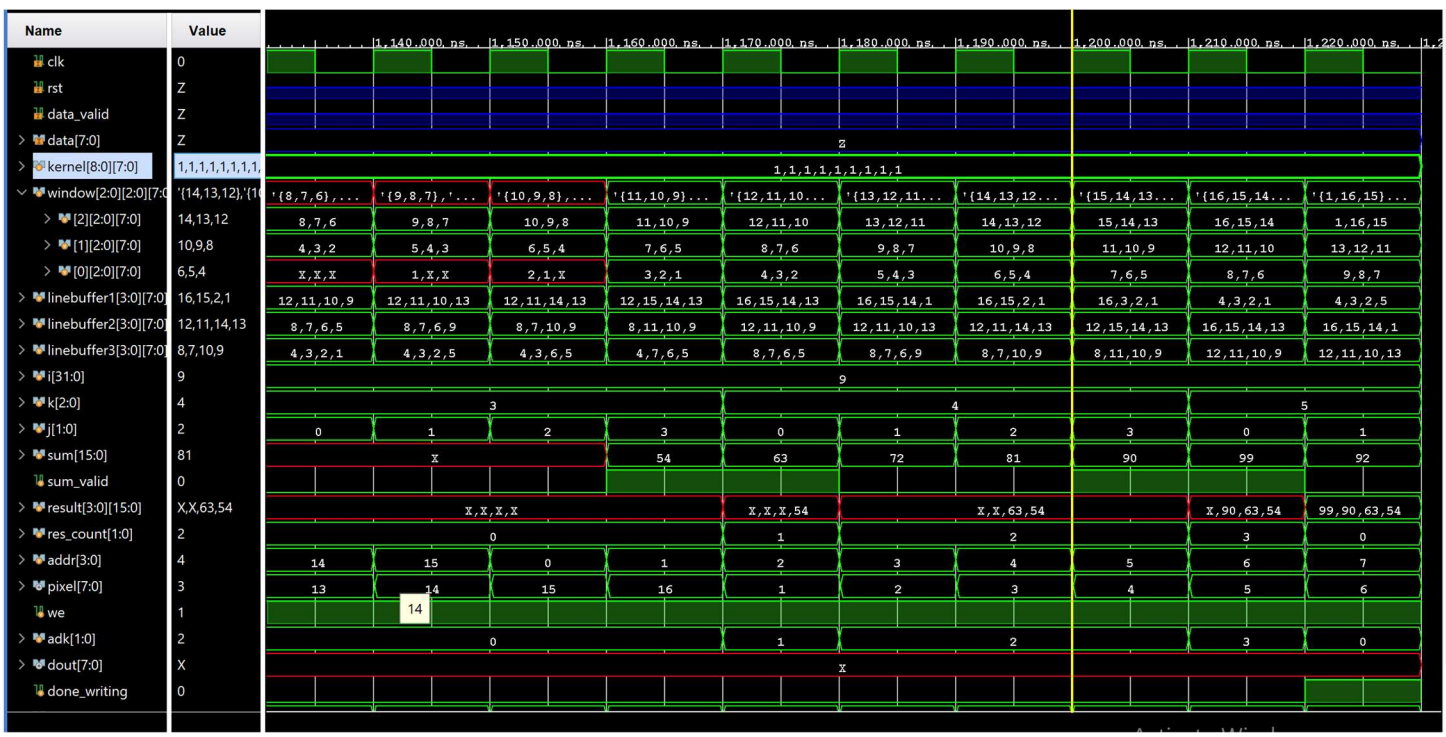
- Multiple MAC operations are performed simultaneously.
- Greatly improves throughput and reduces computation time.

3. Reduced External Memory Access

- Data is passed between neighboring processing elements instead of accessing external memory repeatedly.
- Leads to lower latency and power consumption.

RESULTS

The proposed CNN hardware accelerator has been **designed at the RTL level** and functionally verified through simulation.



Observed Signals and Behavior:

- **Input Signals:**
  - clk shows proper clock operation
  - addr increments sequentially, indicating continuous pixel access from BRAM
  - pixel provides input data stream
- **Line Buffers:**
  - linebuffer1, linebuffer2, and linebuffer3 correctly store previous rows
  - Enable formation of the sliding window

- **Sliding Window:**
  - window[ ][ ] values update correctly every clock cycle
  - Demonstrates proper shifting and loading of new pixel values
- **Computation:**
  - sum shows correct multiply-accumulate (MAC) results
  - Values increase as expected based on input data and kernel
- **Control Signals:**
  - sum_valid is asserted only when valid convolution output is ready
  - addr and internal counters operate correctly
- **Output Storage:**
  - result[] stores computed convolution outputs
  - Data is written only when sum_valid = 1

CONCLUSION

The proposed NeuroSlide CNN hardware accelerator presents an efficient architecture for accelerating convolutional neural network inference.

- The design utilizes pipeline architecture and parallel processing to improve throughput and computational efficiency.
- A combination of line buffers, sliding window mechanism, and parallel MAC operations enables efficient implementation of convolution.
- The use of BRAM-based memory supports high-speed data access and streaming.

The functional simulation results validate the correctness of the design and demonstrate proper data flow through different stages of the architecture.